Spring

Jan Corfixen Sørensen

University of Southern Denmark

Outline

Overview

Spring IoC container

Beans Scope

XML-Based Configuration

Annotation-based Configuration

Java-based Configuration

Summary

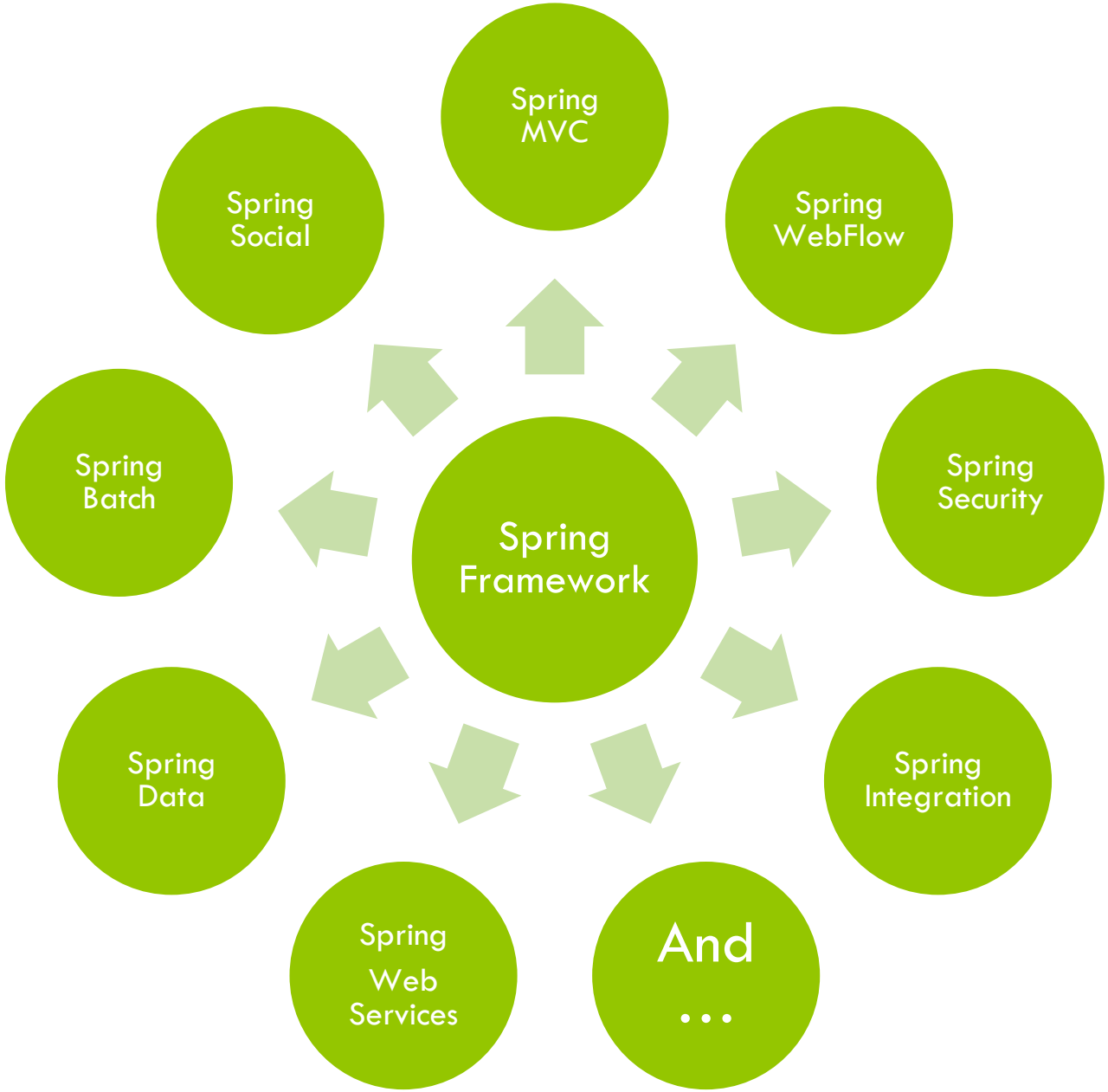
Overview

What is Spring?

- Spring is the most popular application development framework for enterprise Java.

- Millions of developers use Spring to create high performing, easily testable, reusable code without any lock-in.

The Spring Projects



The Spring History

- The first version was written by Rod Johnson
- Expert One-on-One J2EE Design and Development
- First released in June 2003
- Continues Milestone releases
- Awards:
  - Jolt productivity award
  - JAX Innovation award

Why Spring?

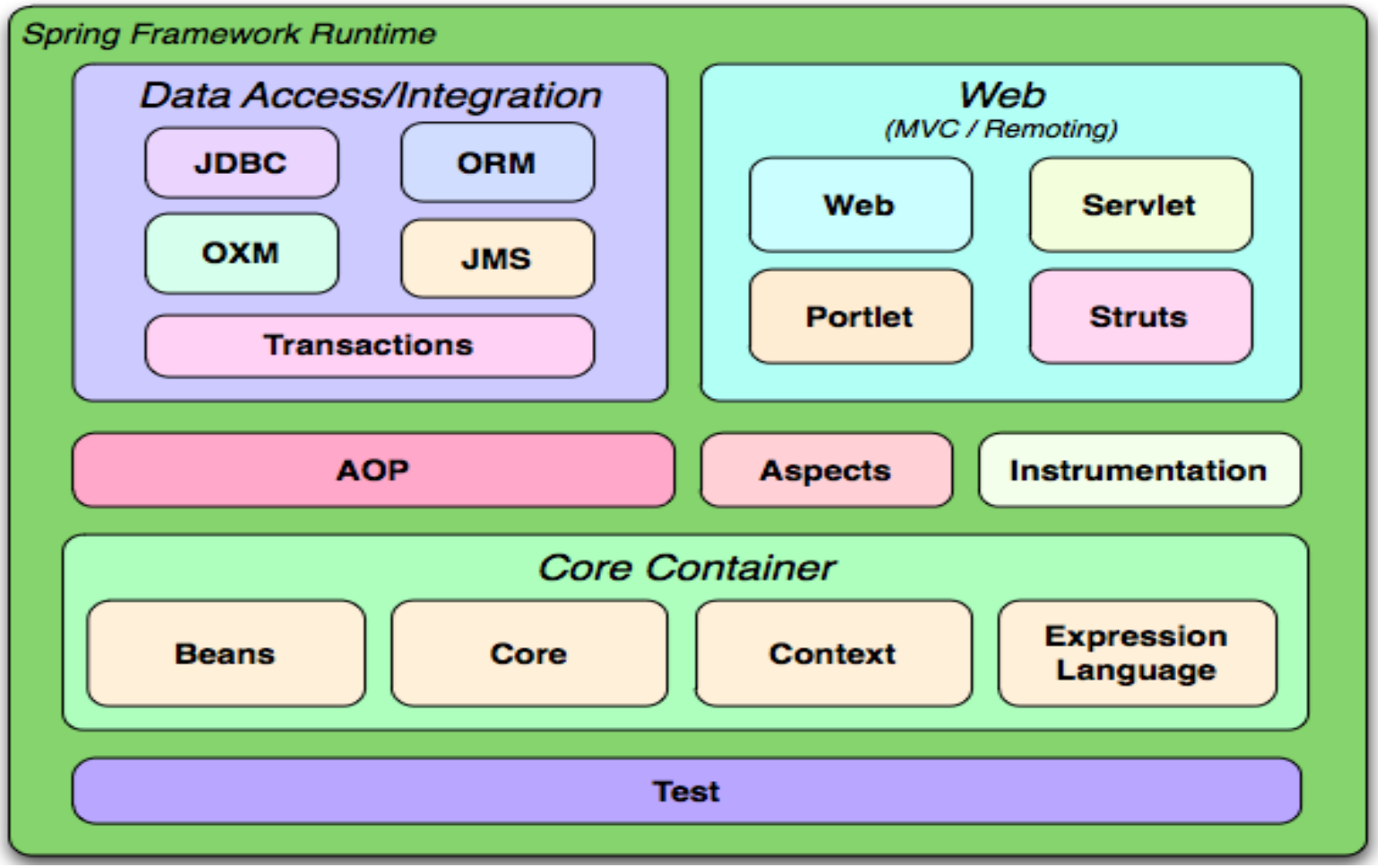
Goals

- Make J2EE easier to use
- Make the common tasks easier
- Promote good programming practice and focus on the domain problems

Today

- An open source application framework and lightweight solution for enterprise applications
- Non-invasive (POJO based), modular and extensible
- De facto standard of J2EE Applications

Spring Modules



Core Container

- Core and Beans
  - provide the fundamental parts of the framework, including IoC and Dependency Injection features
- Context
  - it is a means to access objects in a framework-style manner that is similar to a JNDI registry
- Expression language
  - provides a powerful expression language for querying and manipulating an object graph at runtime

Aspects, Instrumentation

- AOP
  - provides an AOP Alliance-compliant aspect-oriented programming implementation allowing you to define, for example, method-interceptors and pointcuts to cleanly decouple code that implements functionality that should be separated
- Aspect
  - provides integration with AspectJ
- Instrumentation
  - provides class instrumentation support and classloader implementations to be used in certain application servers

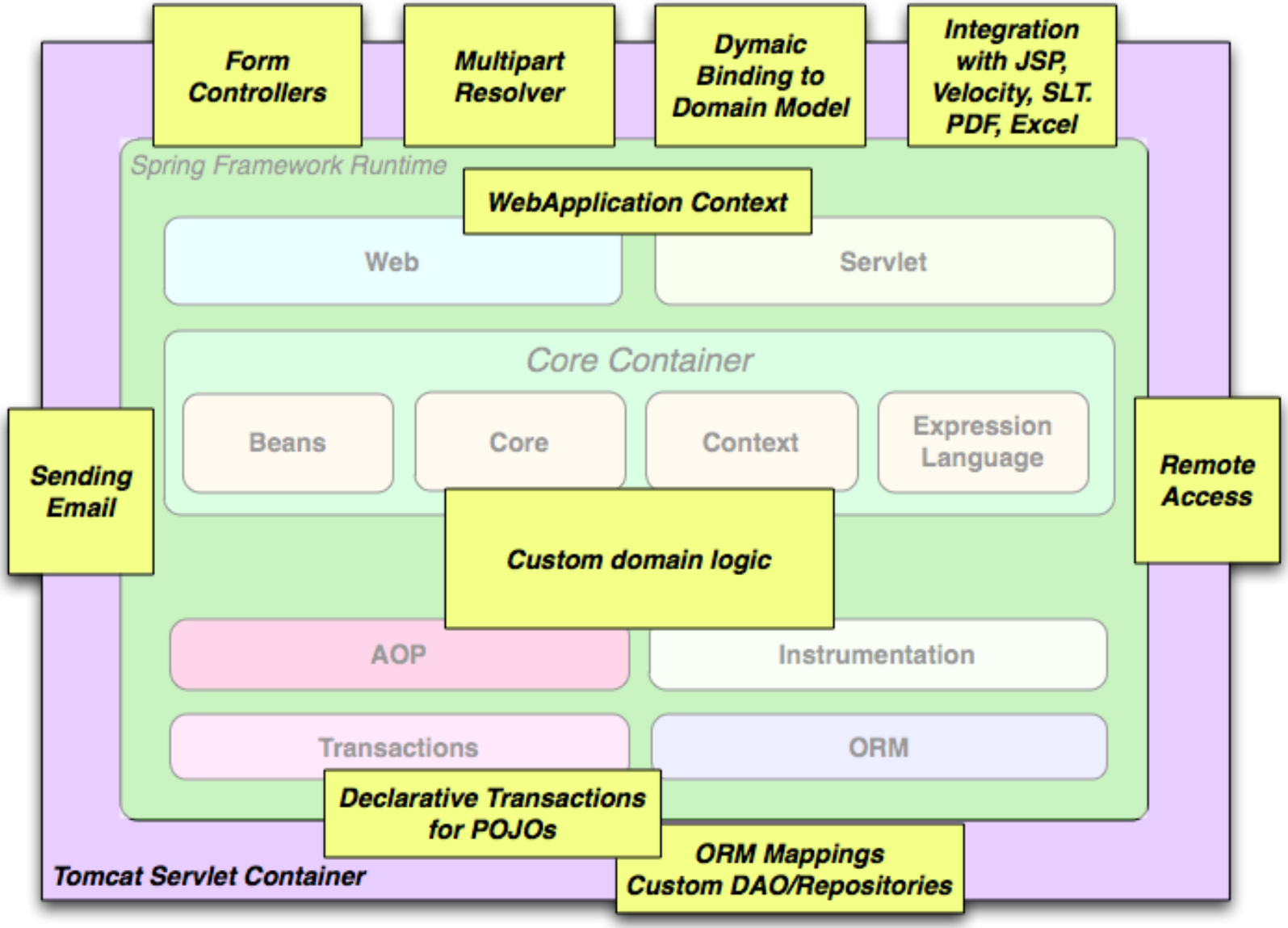
Data Access/Integration

- JDBC
  - provides a JDBC-abstraction layer
- ORM
  - provides integration layers for popular object-relational mapping APIs, including JPA, JDO, Hibernate and iBatis
- OXM
  - provides an abstraction layer that supports Object/XML mapping implementations for JAXB, Castor, XMLBeans, JiBX and XStream.
- JMS
  - contains features for producing and consuming messages.
- Transaction
  - supports programmatic and declarative transaction management.

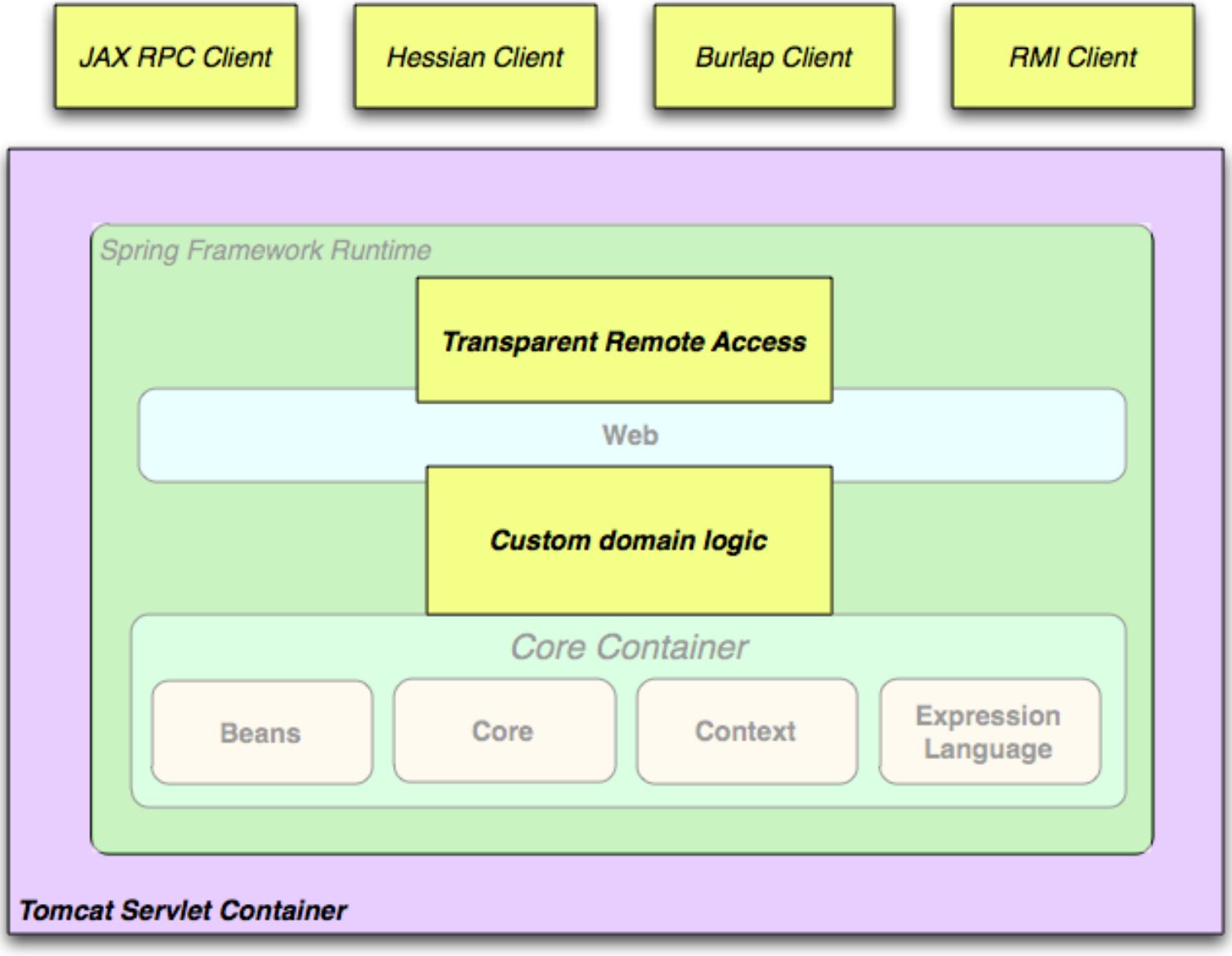
Web

- Spring's WEB
  - provides basic web-oriented integration features
- WEB-Servlet
  - Spring's model-view-controller (MVC) implentation
- WEB-struts
  - contains the classes for integrating a classic Struts WEB tier within a Spring application
- WEB-Portlet
  - provides the MVC implementation to be used in a portlet environment

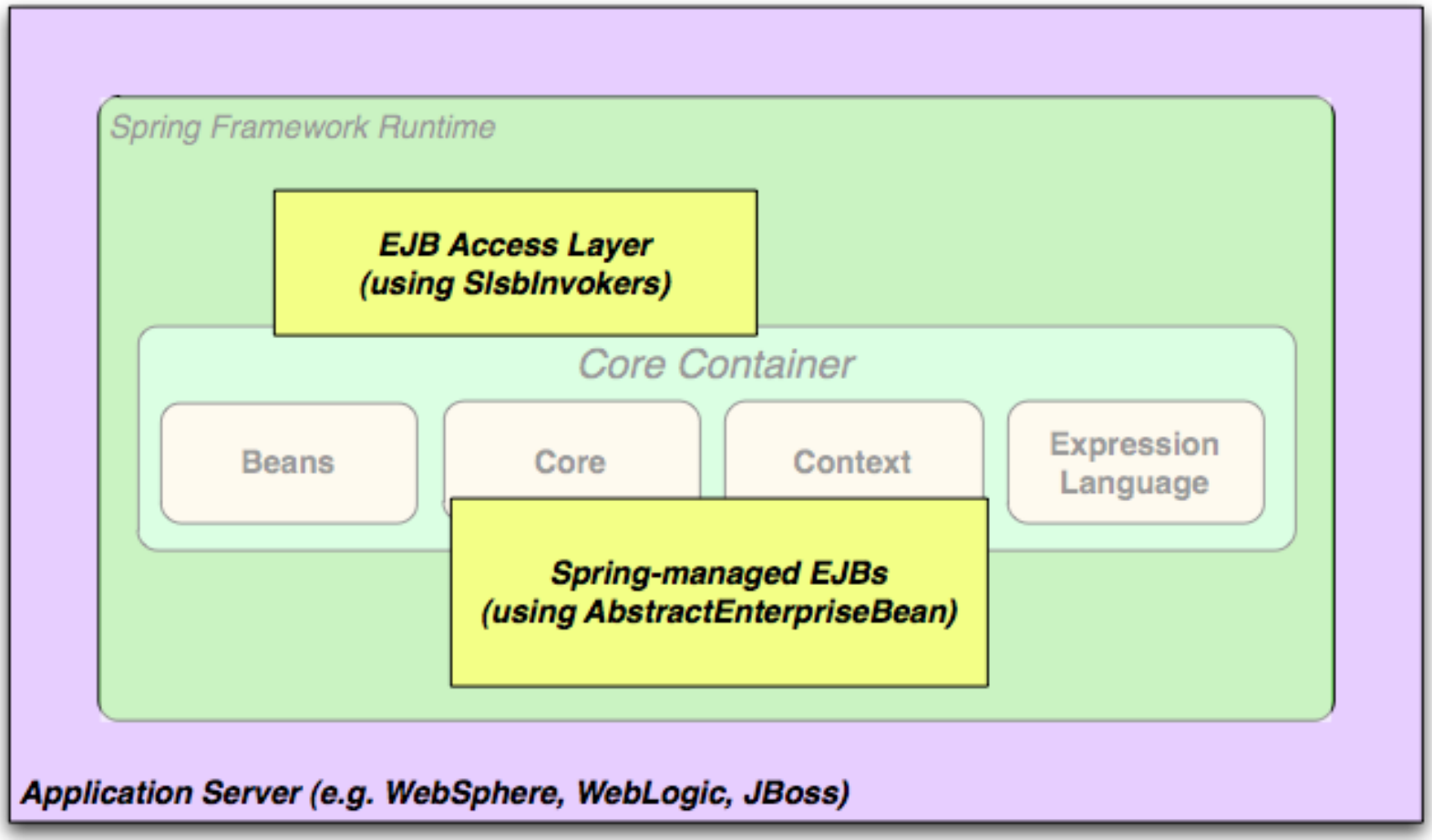
Web Applications



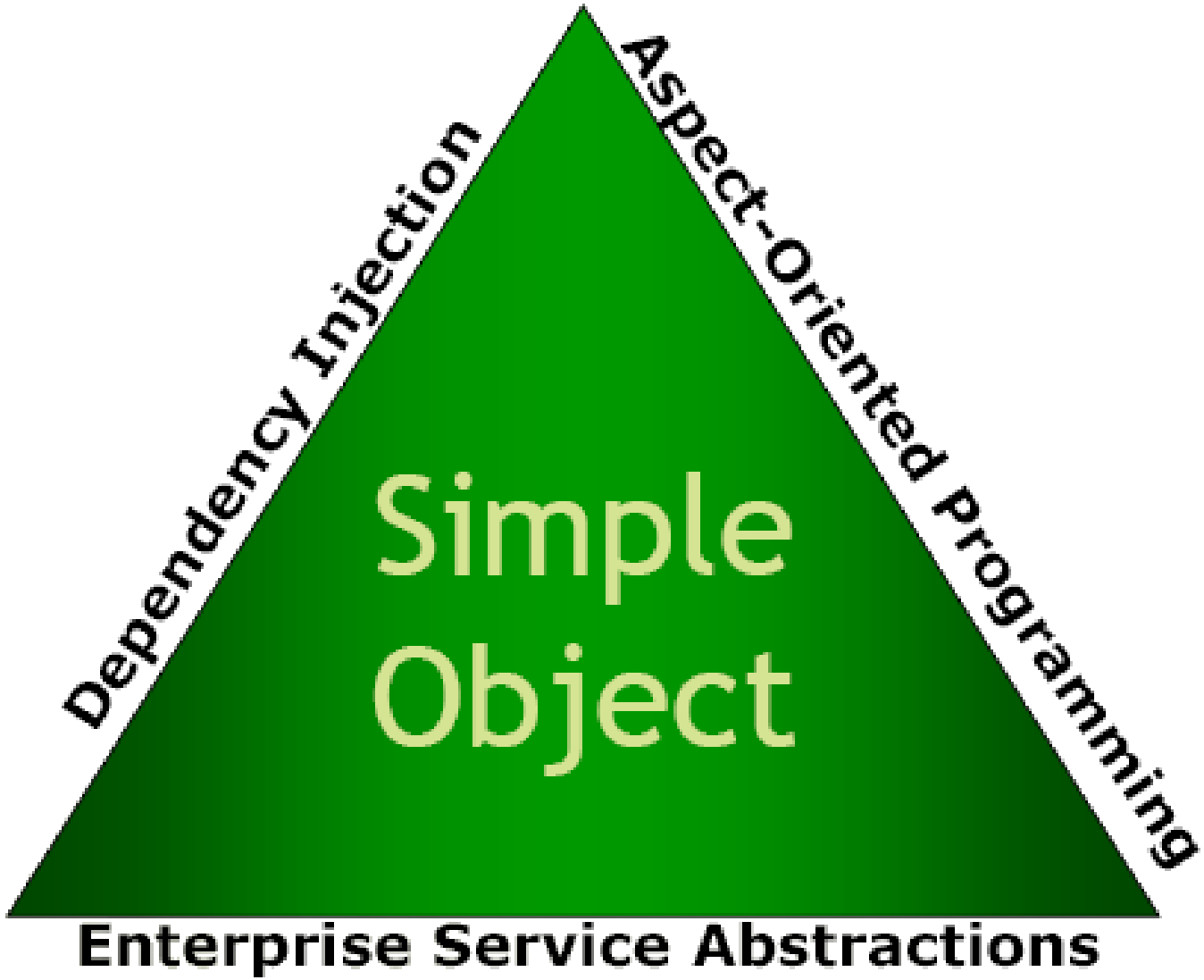
Remoting


JAX RPC Client
Hessian Client
Burlap Client
RMI Client
Spring Framework Runtime
Transparent Remote Access
Web
Custom domain logic
Core Container
Beans
Core
Context
Expression Language
Tomcat Servlet Container

EJB

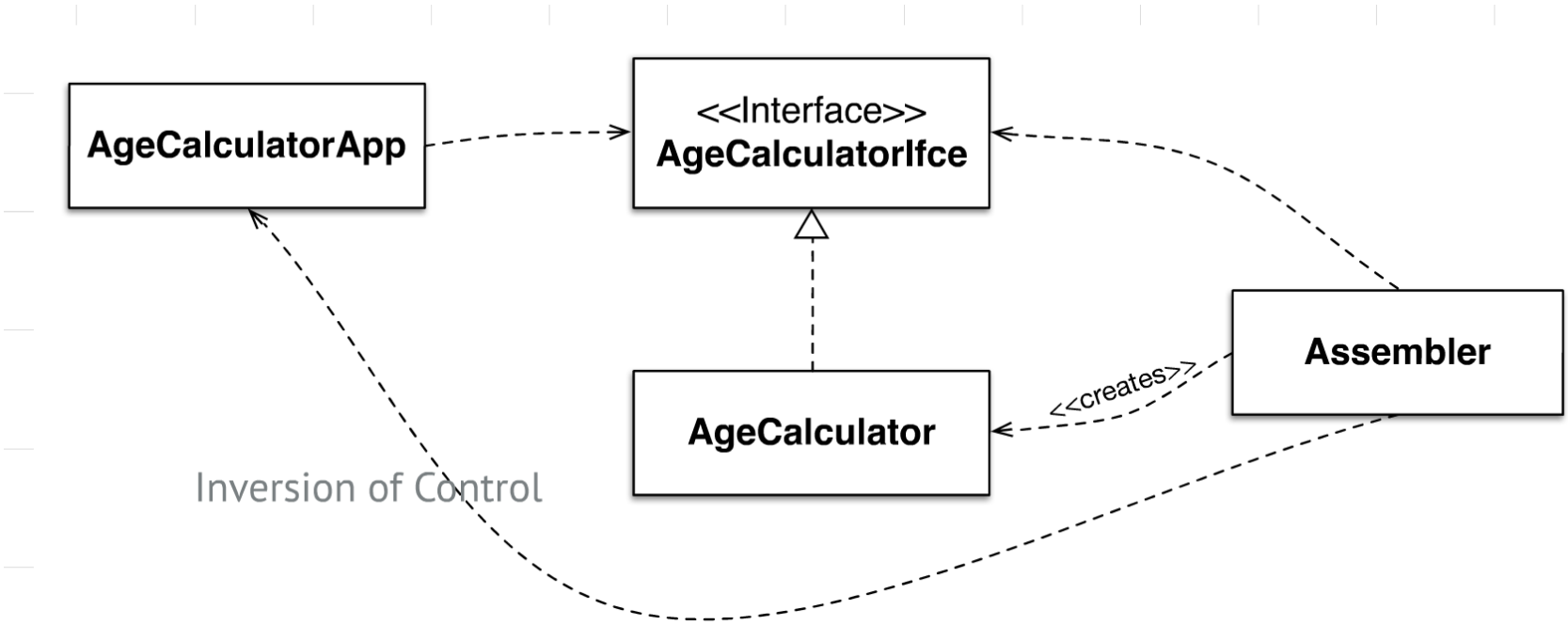

Spring Framework Runtime
EJB Access Layer
(using SlsbInvokers)
Core Container
Beans
Core
Context
Expression
Language
Spring-managed EJBs
(using AbstractEnterpriseBean)
Application Server (e.g. WebSphere, WebLogic, JBoss)

The Spring Triangle



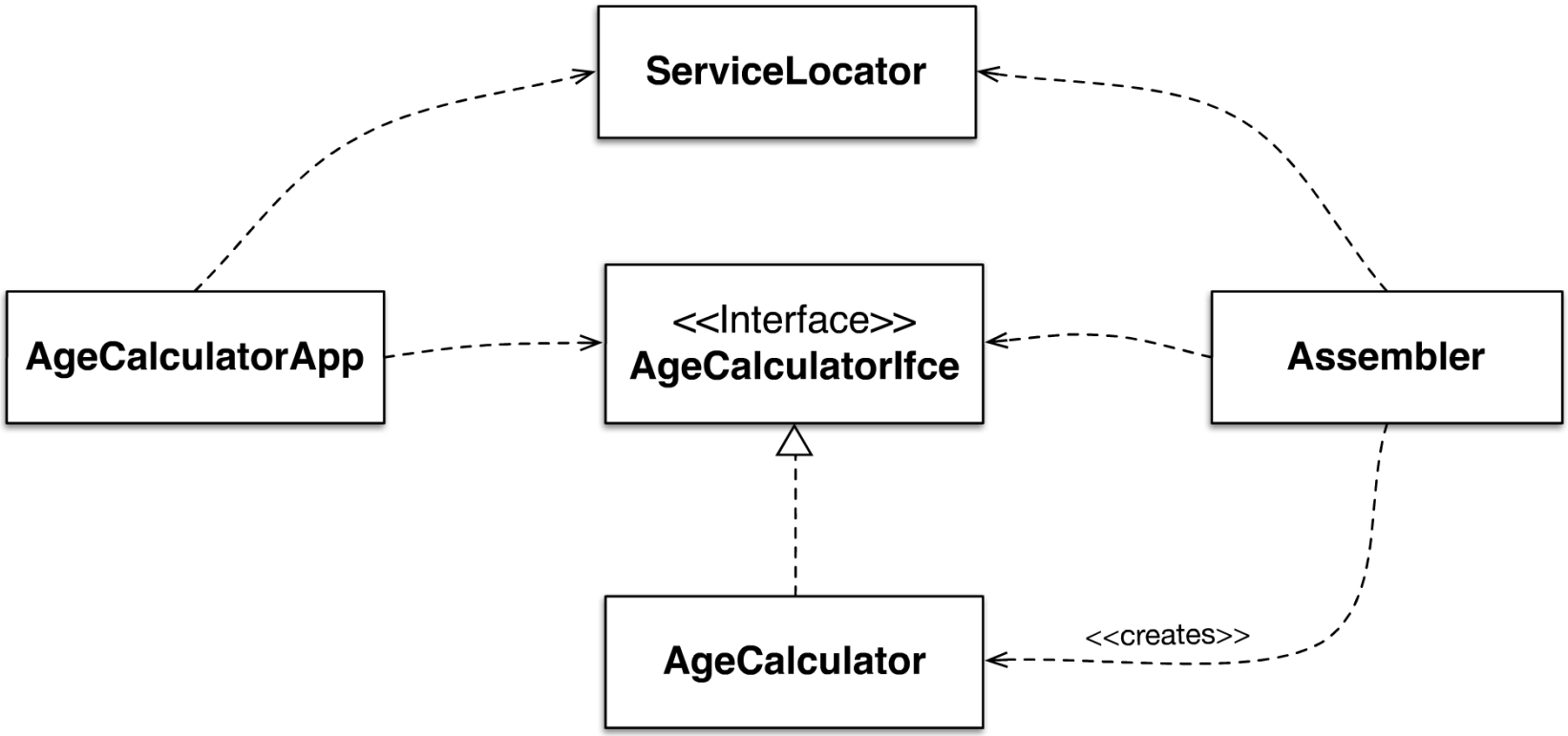
Spring IoC container

What is IoC?



- ”Don’t call me, I’ll call you”
- One form is Dependency Injection (DI)
- No build or run time dependencies

What is Service Locator?



- Whiteboard
- Spring ApplicationContext API
- No build or run time dependencies

Dependency Injection

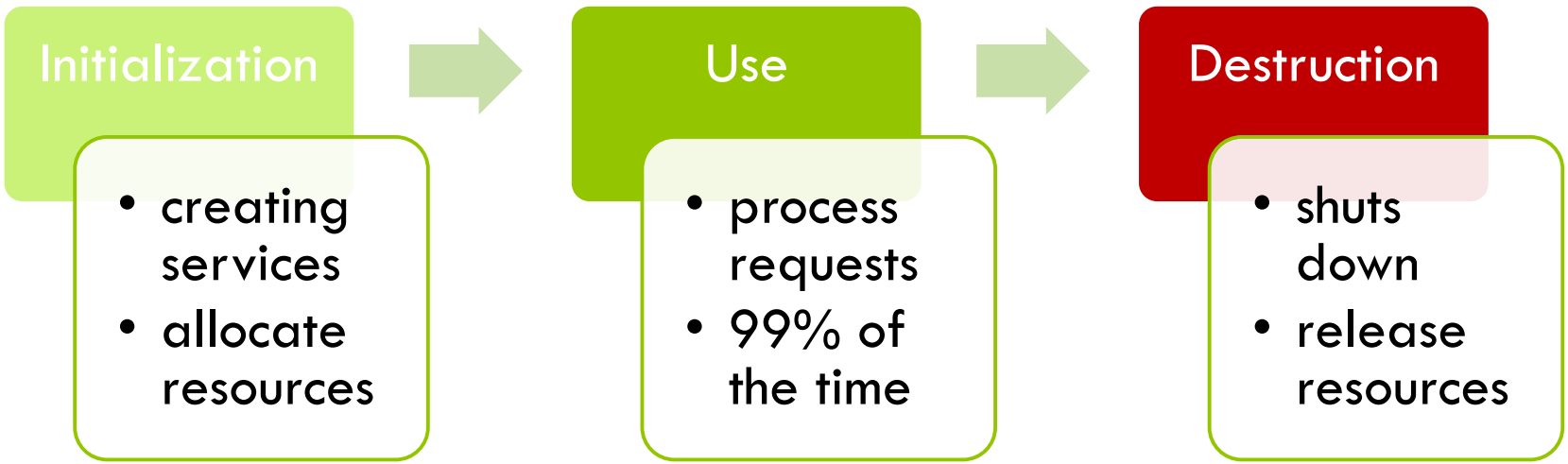
- Is a process whereby objects define their dependencies, that is, the other objects they work with, only through constructor arguments, arguments to a factory method, or properties that are set on the object instance after it is constructed or returned from a factory method.

- Exist in two major variants:
  1. Constructor injection
  2. Setter injection

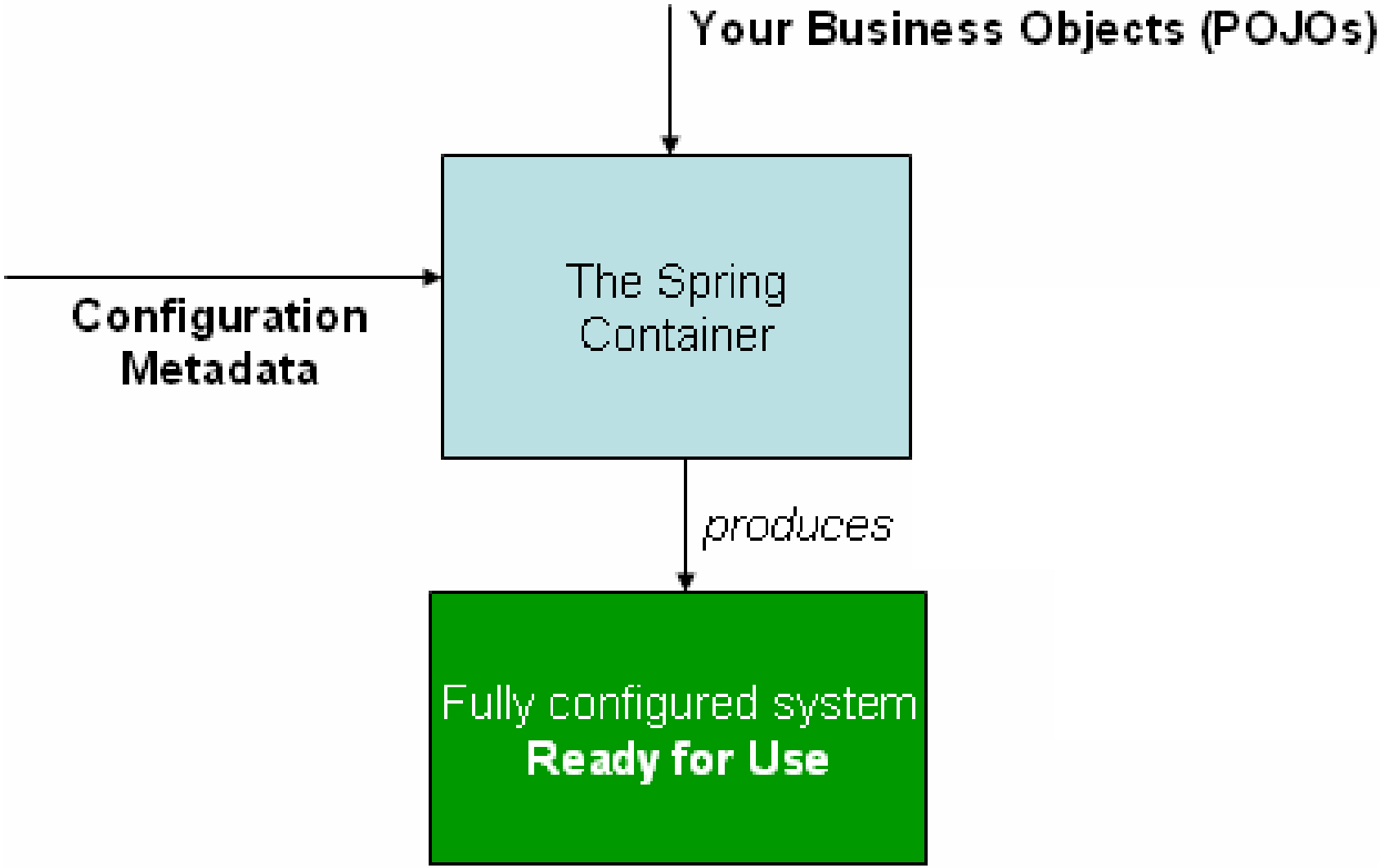
IoC VS DI VS Factory

- DI it is specific type of IoC
- The Factory pattern's main concerns is creating the object graph
- The DI's main concern is how things are connected together
- DI related to Factory and Strategy patterns, but mostly resembles the Build Pattern

Application Lifecycle



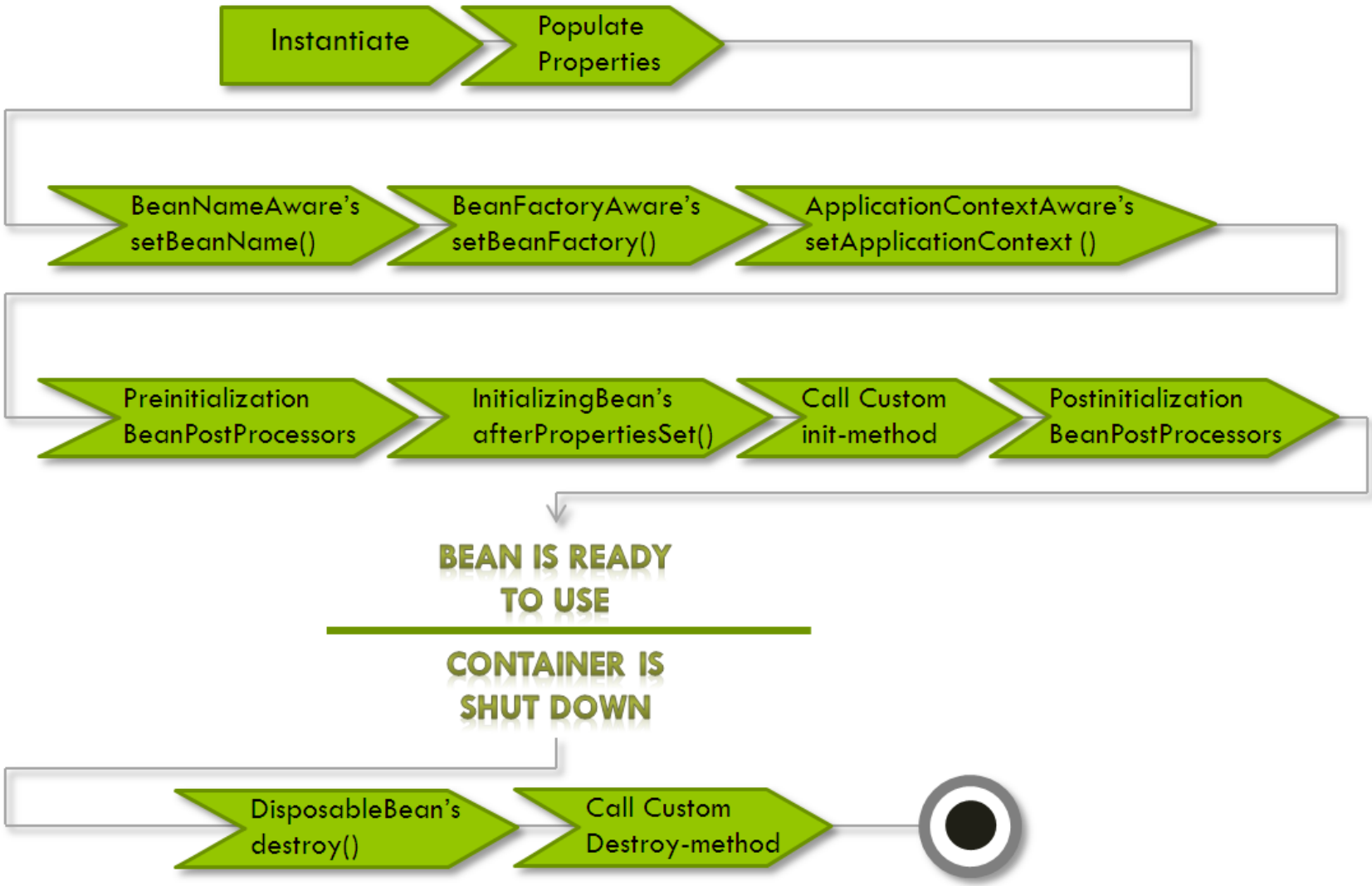
Essence of IoC Container



Terms

- ApplicationContext
  - represents the Spring IoC container
- Bean
  - is an object that managed by Spring IoC container
- BeanDefinition
  - describe a bean instance

Bean Lifecycle



Creating ApplicationContext

- Environments:
  - standalone
  - Web
  - JUnit
  - EJB

- Special prefixes:
  - classpath
  - file system
  - relative path

Standalone Application

- ClassPathApplicationContext
- FileSystemApplicationContext

```
ApplicationContext context =
  new ClassPathXmlApplicationContext(
    new String[] {"services.xml", "repositories.xml"});

// retrieve configured instance
CurrencyService currencyService
       = context.getBean("currency", CurrencyService.class);

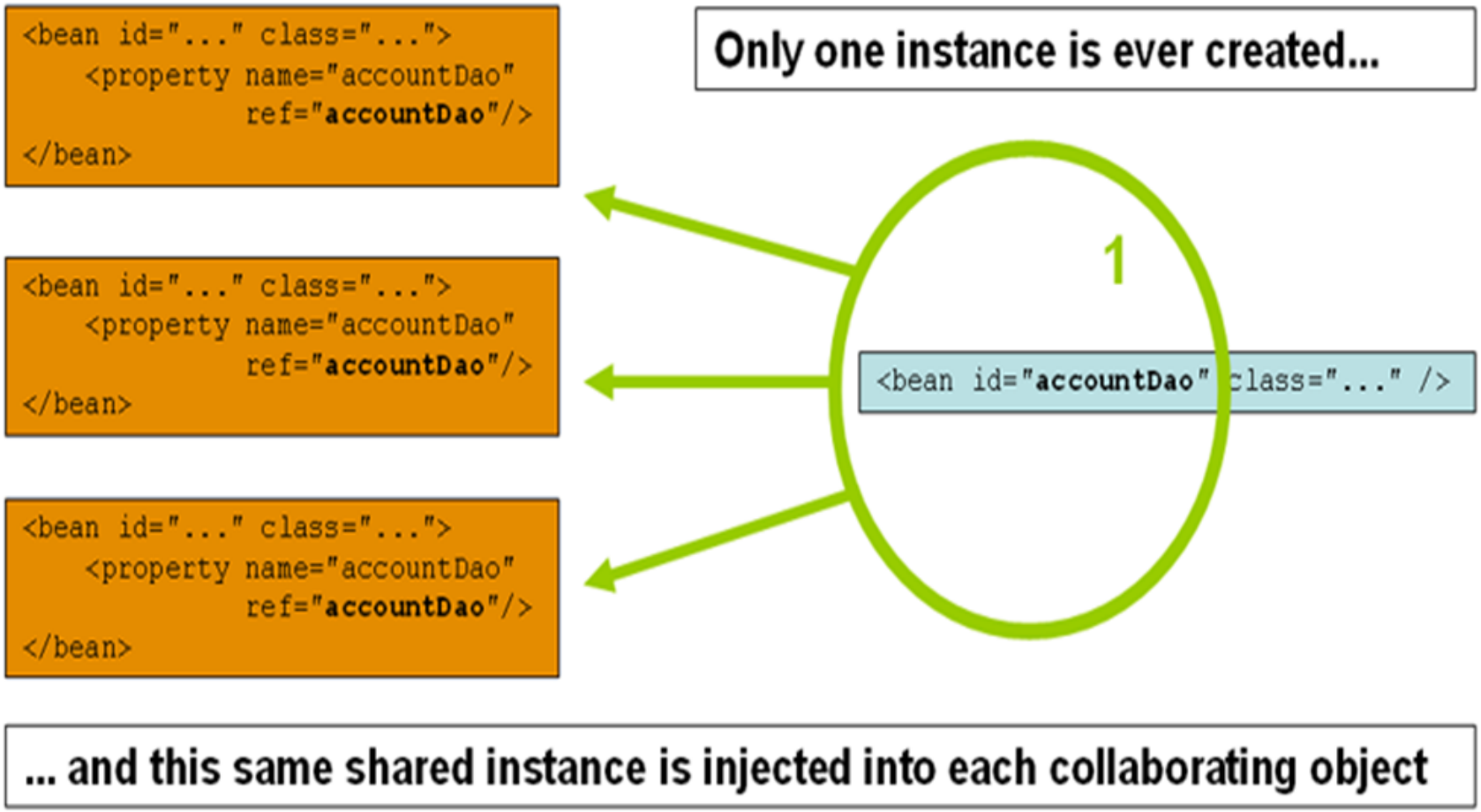
AccountService accountService
       = context.getBean(AccountService.class);
```

Beans Scope

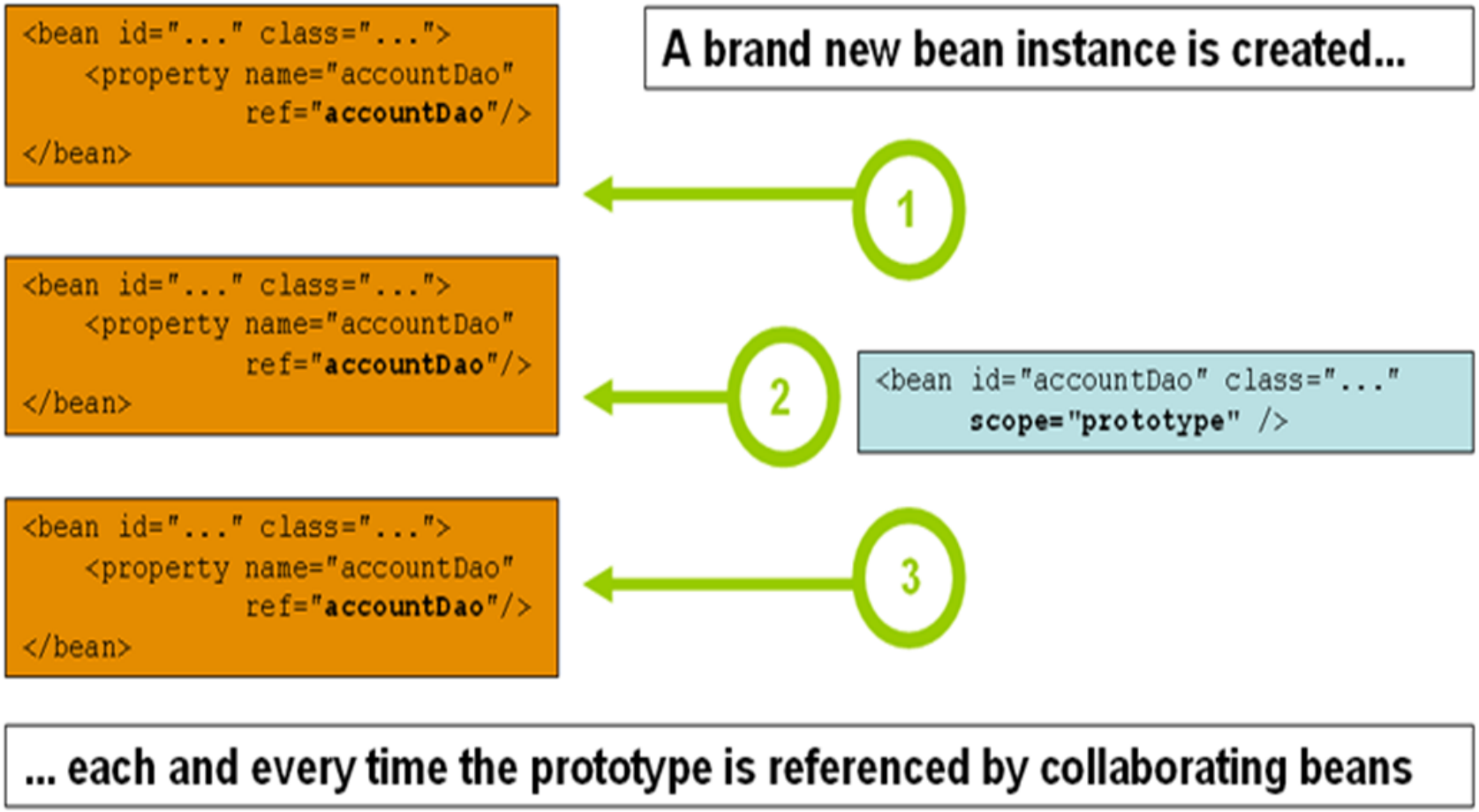
Spring Beans Scopes

- simple
  - singleton
  - prototype
- runtime
  - thread
  - custom impl.
- web-aware (only for web)
  - request
  - session
  - global session

The Singleton Scope



The Prototype Scope



XML-Based Configuration

Namespaces

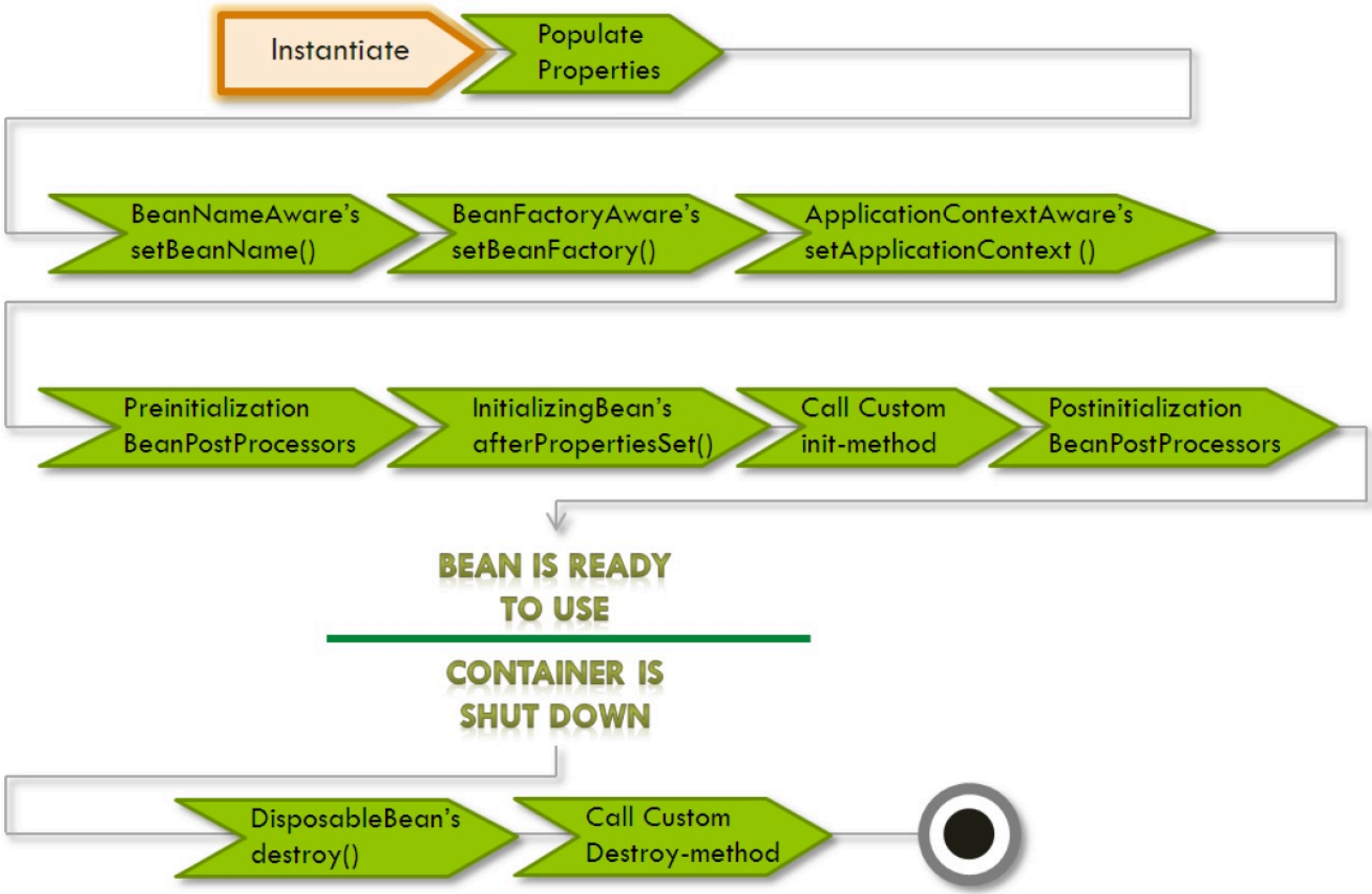
```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
           http://www.springframework.org/schema/aop
           http://www.springframework.org/schema/aop/spring-aop-3.0.xsd
           http://www.springframework.org/schema/context
           http://www.springframework.org/schema/context/spring-context-3.0.xsd">

  <bean id="" class=""></bean>

  <bean id="" class=""/>
</beans>
```

Instantiate Bean



Naming Beans

```
<beans>
  <bean id="beanId" class="org.exhanger.api.OrderServiceImpl"/>

  <bean name="name/" class="org.exhanger.api.OrderServiceImpl"/>

  <bean name="name1,name2;name3 name4" class="...">
  </bean>

  <alias name="fromName" alias="toName"/>
</beans>
```

Instantiate Bean

- with a constructor
- with a static factory method
- using an instance factory method
- with the FactoryBean

Constructor Instantiate

- simple class

```
<bean id="exampleBean" class="examples.ExampleBean"/>
public class ExampleBean {
  public ExampleBean() {}
}
```

- inner class

```
<bean id="innerBean" class="examples.ExampleBean$InnerBean"/>
public class ExampleBean {
  public static class InnerBean {
  }
}
```

Static Factory Method

```
<bean id="clientService" class="examples.ClientService"
      factory-method="createInstance"/>
```

```
public class ClientService {
  private static ClientService clientService = new ClientService();

  private ClientService() {}

  public static ClientService createInstance() {
    return clientService;
  }
}
```

Instance Factory Method

```
<!-- the factory bean-->
<bean id="serviceLocator" class="example.ServiceLocator"/>
<!-- the bean to be created via the factory bean -->
<bean id="clientService"
      factory-bean="serviceLocator"
      factory-method="createClientServiceInstance"/>
```

```
public class ServiceLocator {
  private static ClientService clientService = new ClientService();

  public ClientService createClientServiceInstance() {
    return clientService;
  }
}
```

Factory Bean

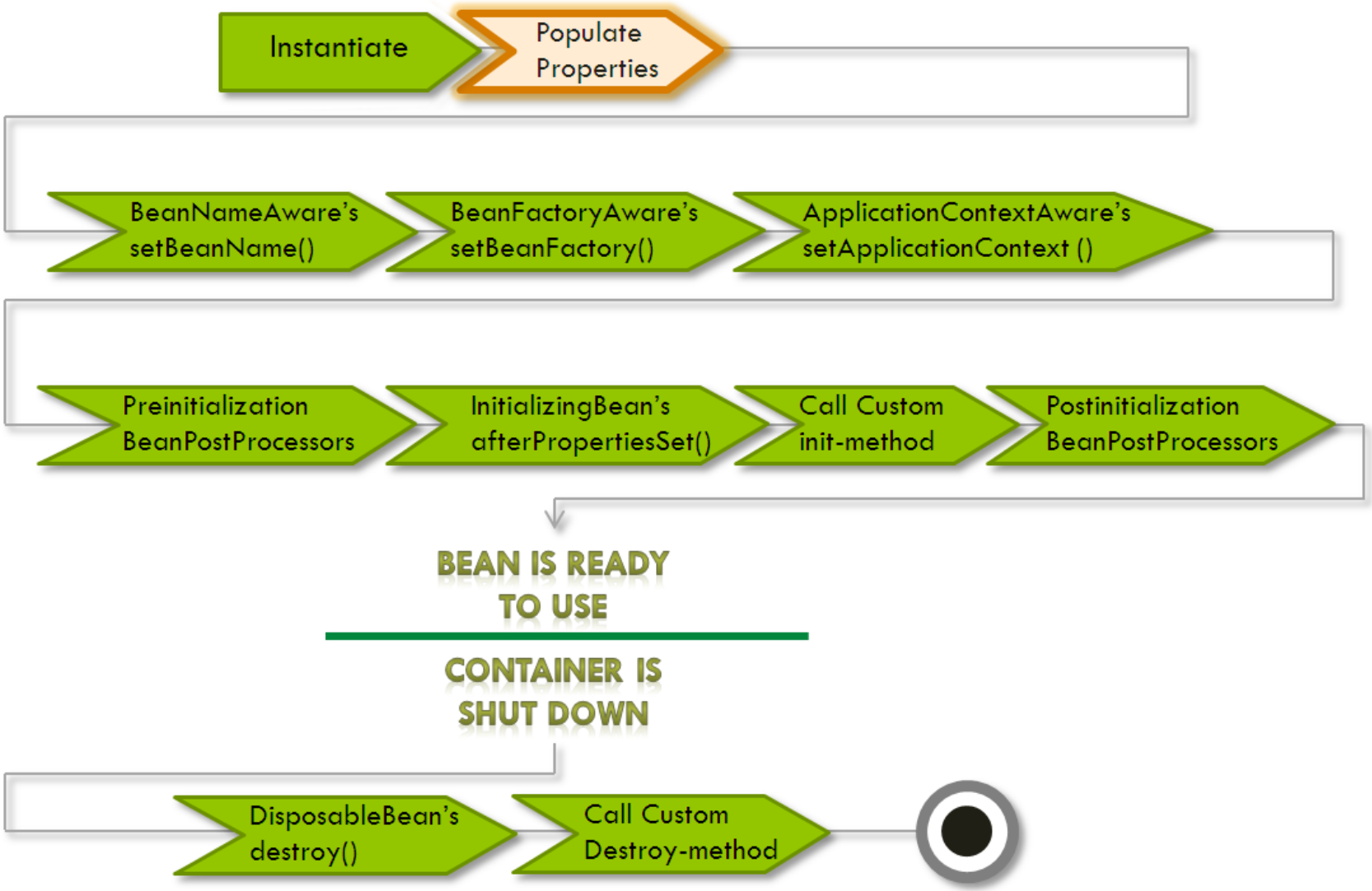
```
public interface FactoryBean<T> {
  /**Return an instance of the object managed by this factory.*/
  T getObject() throws Exception;

  /** Return the type of object that this FactoryBean creates.*/
  Class<?> getObjectType();

  /**Is the object managed by this factory a singleton? */
  boolean isSingleton();
}
```

```
<bean id="bean" class="FactoryBean"/>
```

Dependency Injection



Constructor-based DI

- is accomplished by the container invoking a **constructor** with a number of arguments, each representing a dependency
- calling a static factory **method** with specific arguments to construct the bean is nearly equivalent

Constructor argument resolution

- argument resolution

```
<bean id="b1" class="org.examples.ClientServiceBean">
  <constructor-arg ref="b2"/>
  <constructor-arg ref="b3"/>
</bean>
```

- argument type matching

```
<bean id="b2" class="ClientService" factory-method="getInstance">
  <constructor-arg type="int" value="7500000"/>
  <constructor-arg type="java.lang.String" value="42"/>
</bean>
```

- argument index

```
<bean id="b3" factory-bean="factory" factory-method="getInstance"/>
  <constructor-arg index="0" value="75"/>
  <constructor-arg index="1" value="42"/>
</bean>
```

Setter-based DI

- is accomplished by the container calling **setter methods** on your beans after invoking a no-argument constructor or no-argument static factory method to instantiate bean

Constructor VS Setter

- constructor
  - mandatory dependencies
  - immutability
- setter
  - optional dependencies and default values
  - obvious names
  - auto inheritance

Straight Values

```
<bean id="dataSource"
      class="org.apache.commons.dbcp.BasicDataSource" >

  <!-- results in a setDriverClassName(String) call -->
  <property name="driverClassName" value="com.mysql.jdbc.Driver"/>
  <property name="url">
      <value>jdbc:mysql://localhost:3306/mydb</value>
  </property>
  <property name="username" value="root"/>
  <property name="password" value="masterkaoli"/>
</bean>
```

Null and Empty

```
<!-- is equivalent to the following code: bean.setEmail("").-->
<bean class="Bean">
  <property name="email" value=""/>
</bean>
```

```
<!-- The <null/> element handles null values. -->
<bean class="Bean">
  <property name="email"><null/></property>
</bean>
```

Properties

```
<bean class="org.springframework.beans.factory.config.PropertyPlaceholderConfigurer">
  <property name="locations" value="classpath:jdbc.properties"/>
</bean>

<bean id="dataSource" destroy-method="close"
      class="org.apache.commons.dbcp.BasicDataSource">
  <property name="driverClassName" value="${jdbc.driverClass}"/>
  <property name="url" >
    <value>${jdbc.url}</value>
  </property>
  <property name="username" value="${jdbc.username}"/>
  <property name="password" value="${jdbc.password}"/>
</bean>
```

Bean References

- bean

```
<ref bean="someBean"/>
```

- local

```
<ref local="someBean"/>
```

- parent

```
<!-- in the parent context -->
<bean id="accountService" class="com.foo.SimpleAccountService"/>

<!-- in the child (descendant) context -->
<bean id="accountService" class="org.springframework.ProxyFactoryBean">
  <property name="target">
    <ref parent="accountService"/>
  </property>
</bean>
```

Inner Bean

```
<bean id="orderService" class="OrderServiceImpl">
  <property name="repository" ref="orderRepository"/>
  <property name="builder">
    <bean class="OrderBuilderImpl">
      <constructor-arg ref="accountService"/>
      <constructor-arg ref="currencyService"/>
    </bean>
  </property>
</bean>
```

Collections (1)

```
<!-- setAdminEmails(java.util.Properties) call -->
<property name="adminEmails">
  <props>
    <prop key="administrator">administrator@gmail.com</prop>
  </props>
</property>
<!-- setSomeList(java.util.List) call -->
<property name="someList">
  <list>
    <value>a list element followed by a reference</value>
    <ref bean="myDataSource" />
  </list>
</property>
```

Collections (2)

```
<!-- setSomeMap(java.util.Map) call -->
<property name="someMap">
  <map>
    <entry key="an entry" value="just some string"/>
    <entry key ="a ref" value-ref="myDataSource"/>
  </map>
</property>
<!-- setSomeSet(java.util.Set) call -->
<property name="someSet">
  <set>
    <value>just some string</value>
    <ref bean="myDataSource" />
  </set>
</property>
```

Importing Configuration Files

- xml-context-config.xml

```
<beans>
  <import resource="classpath:xml-repository-config.xml"/>
  <import resource="classpath:xml-service-config.xml"/>
</beans>
```

- xml-service-config.xml

```
<beans>
  <bean id="currencyRepository" class="CurrencyMapRepository"/>
</beans>
```

- xml-repository-config.xml

```
<bean id="currencyService" class="CurrencyServiceImpl">
  <constructor-arg ref="currencyRepository"/>
</bean>
```

Lazy-initialized Beans

- IoC container create a bean instance when it is first requested

```
<bean id="lazy" class="ExpensiveToCreateBean" lazy-init="true"/>
```

- control lazy-initialization at the container level

```
<beans default-lazy-init="true">
  <!-- no beans will be pre-instantiated... -->
</beans>
```

- disadvantages
  - errors in the configuration or surrounding
  - environment are detected some time later

Autowiring

- advantages
  - can significantly **reduce** the need to **specify** properties or constructor arguments
  - can **update** a configuration as your objects evolve
- disadvantages
  - cannot autowire **simple** properties (primitives)
  - autowiring is **less exact** than explicit wiring
  - wiring information may not be available to tools that may generate documentation from a Spring container

Autowiring Modes

- no (default)
- byName

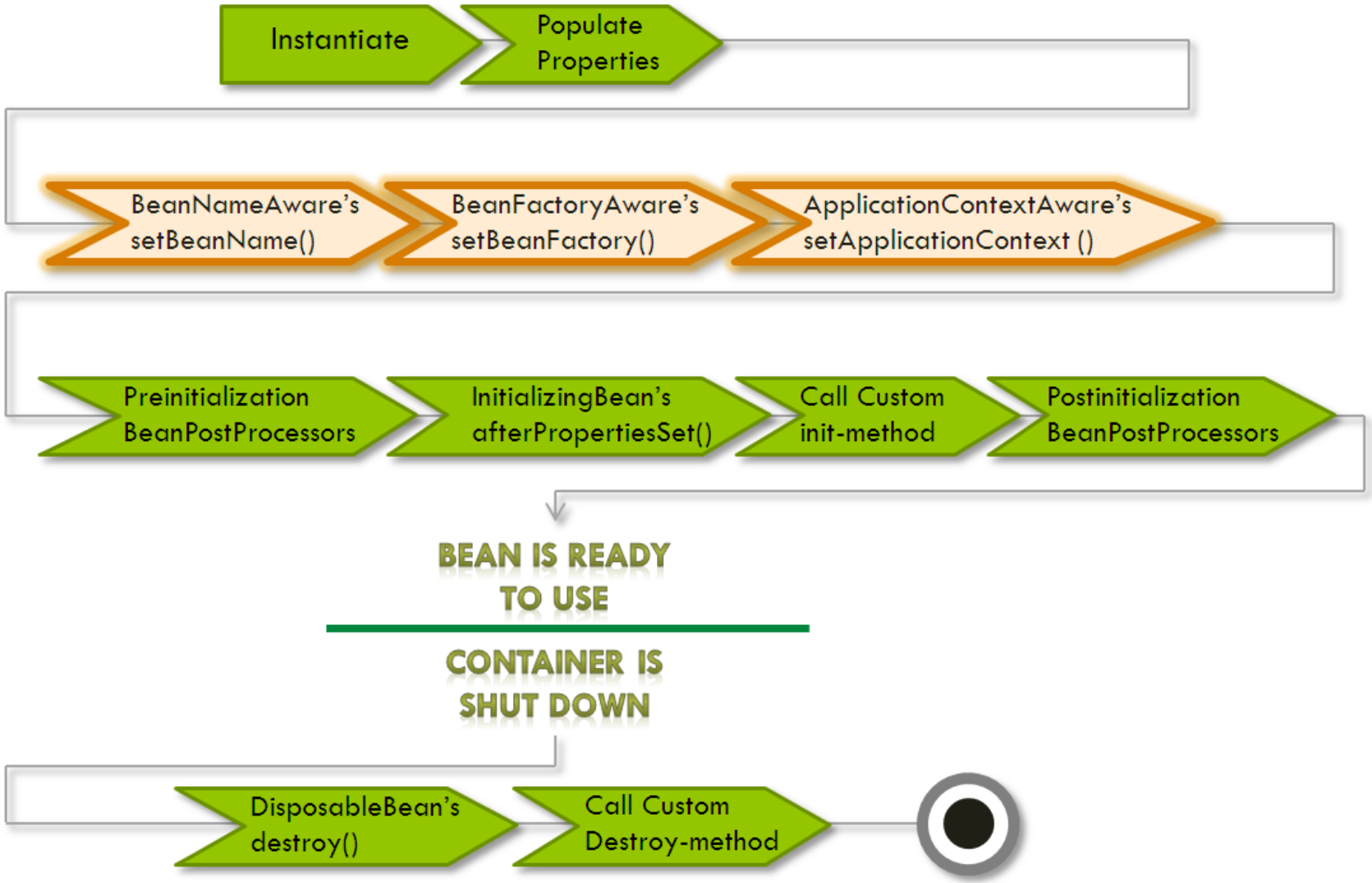
```
<bean id="messageSource" class="MessageSourceImpl" />
public void setMessageSource(MessageSource messageSource)
```

- byType

```
<bean id="messageSource" class="MessageSourceImpl" />
public void setMessageSource(MessageSource messageSource)
```

- constructor
- autodetect

Spring Aware Interfaces



Spring Aware Interfaces

- BeanNameAware
- BeanFactoryAware
- ApplicationContextAware

- ApplicationEventPublisherAware
- ServletConfigAware
- ServletContextAware
- MessageSourceAware

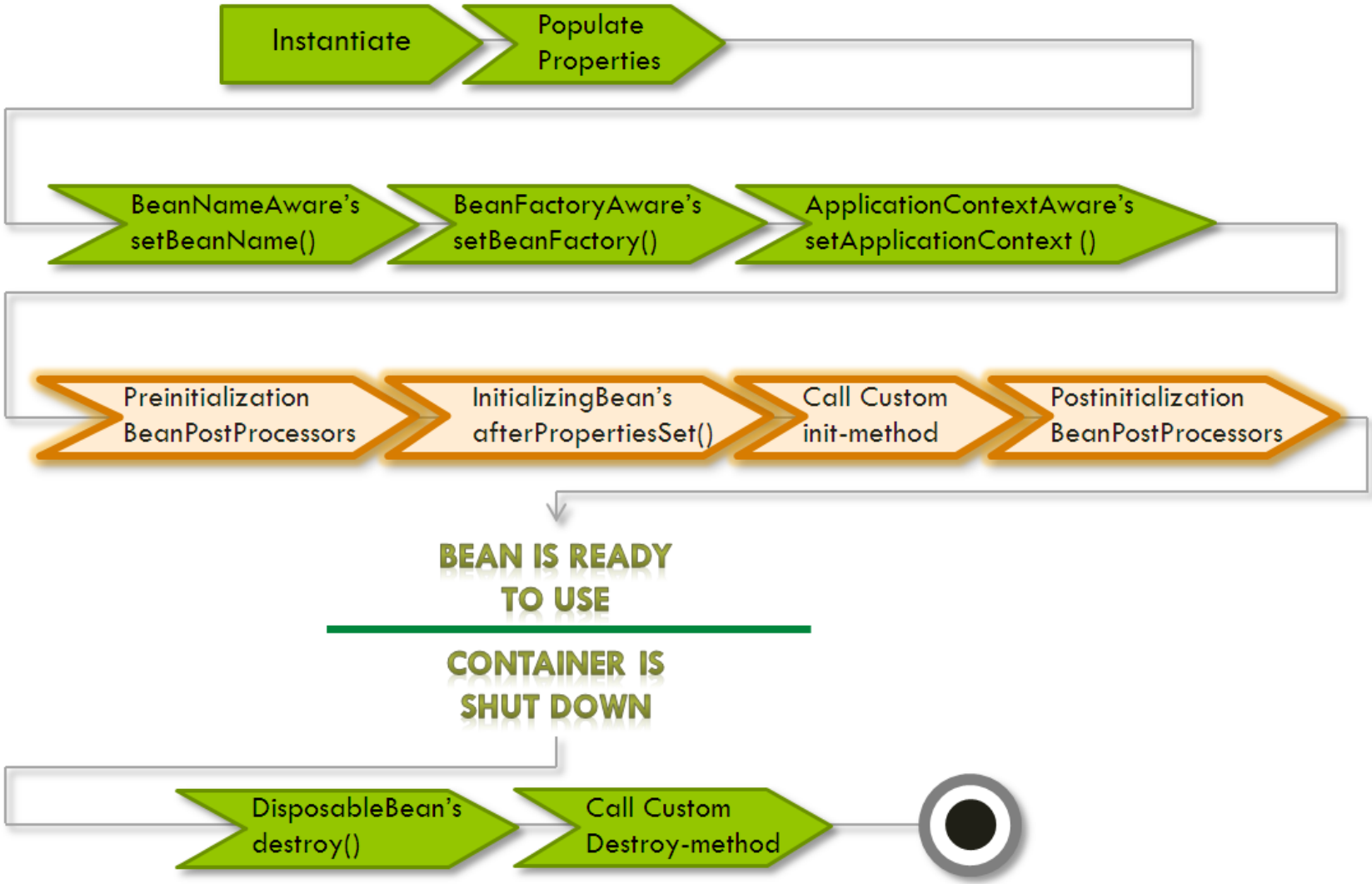
Application Events

```
public class SomeService implements ApplicationContextAware {
    public void setApplicationContext(ApplicationContext ctx) {}

    public void anyMethod(String value) {
       context.publishEvent(new AnyEvent(value));
    }
}

public class Listener implements ApplicationListener<AnyEvent> {
    public void onApplicationEvent(AnyEvent event) {
    }
}
```

Bean Initialization



Bean Post Processor

```
/**
* ApplicationContexts can autodetect BeanPostProcessor beans in their
* bean definitions and apply them to any beans subsequently created. */
public interface BeanPostProcessor {
/**
* Apply this BeanPostProcessor to the given new bean instance before any bean
* initialization callbacks (like afterPropertiesSet or a custom init-method)*/
Object postProcessBeforeInitialization(Object bean, String beanName) throws BeansException

/**
* Apply this BeanPostProcessor to the given new bean instance after any bean
* initialization callbacks (like afterPropertiesSet or a custom init-method).
*/
Object postProcessAfterInitialization(Object bean, String beanName) throws BeansException;

}
```

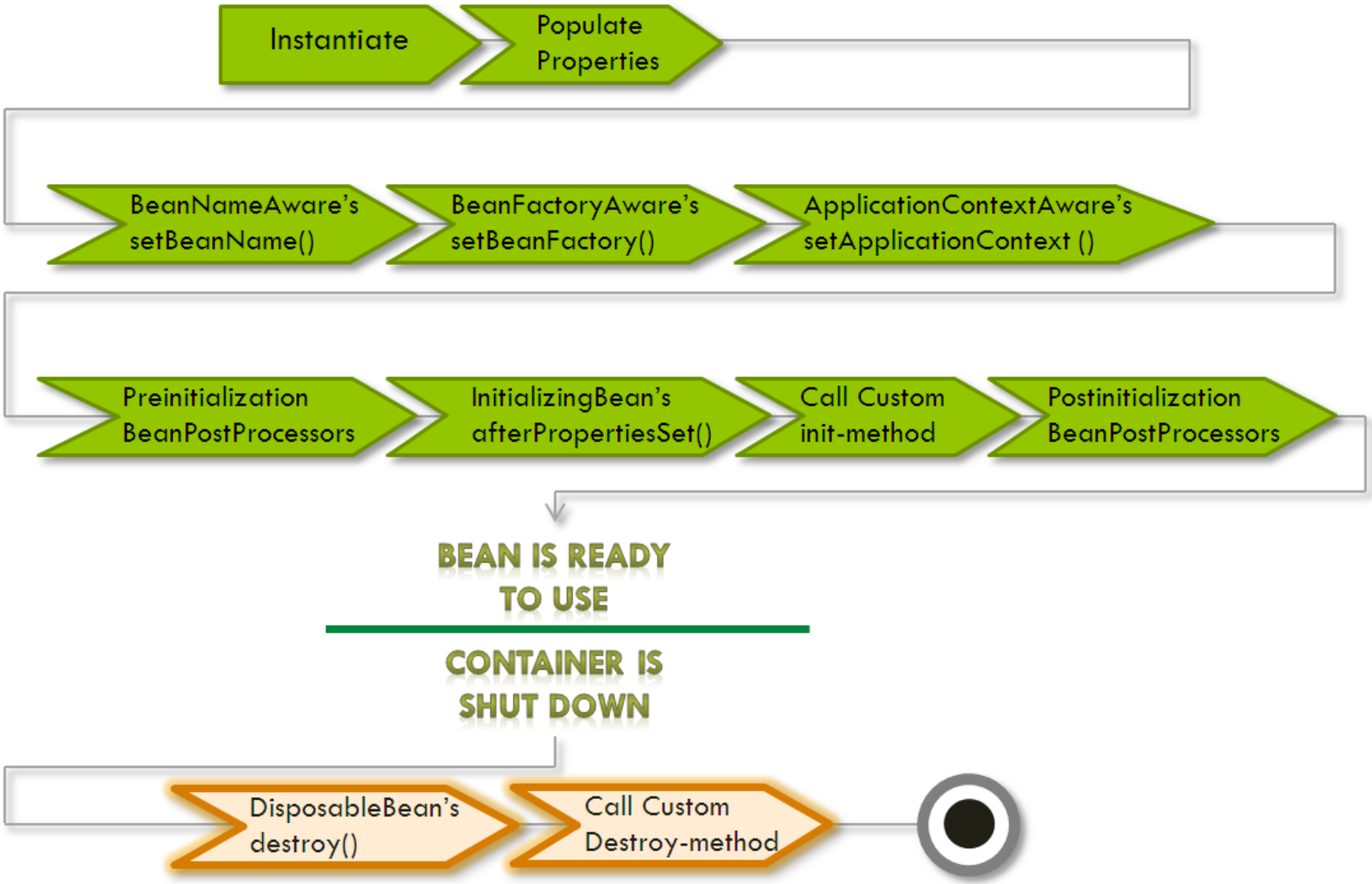
Initialization Callbacks

```
<bean id="bean1" class="exampl.ExampleBean1" init-method="init"/>
public class ExampleBean1 {
  public void init() {}
}

<bean id="bean2" class="examples.ExampleBean2"/>
public class ExampleBean2 implements InitializingBean {
  public void afterPropertiesSet() {}
}

public class ExampleBean2 {
  @PostConstruct
  public void initialize() {}
}
```

Destroy Beans



Destruction Callbacks

```
<bean id="bean1" class="example.Bean1" destroy-method="cleanup"/>
public class Bean1 {
  public void cleanup() {}
}

<bean id="bean2" class="example.Bean2"/>
public class Bean2 implements DisposableBean {
  public void destroy() {}
}

public class Bean2 {
  @PreDestroy
  public void destroy() {}
}
```

Combining Lifecycle Mechanisms

- initialization methods, are called as follows:
  - methods annotated with `@PostConstruct`
  - afterPropertiesSet as defined by **InitializingBean**
  - **custom** configured init method
- destroy methods are called in the same order:
  - method annotated with `@PreDestroy`
  - destroy() as defined by the **DisposableBean**
  - **custom** configured destroy method

Annotation-based Configuration

Annotation-based Configuration

- an **alternative** to XML setups
- annotation injections is performed **before** XML

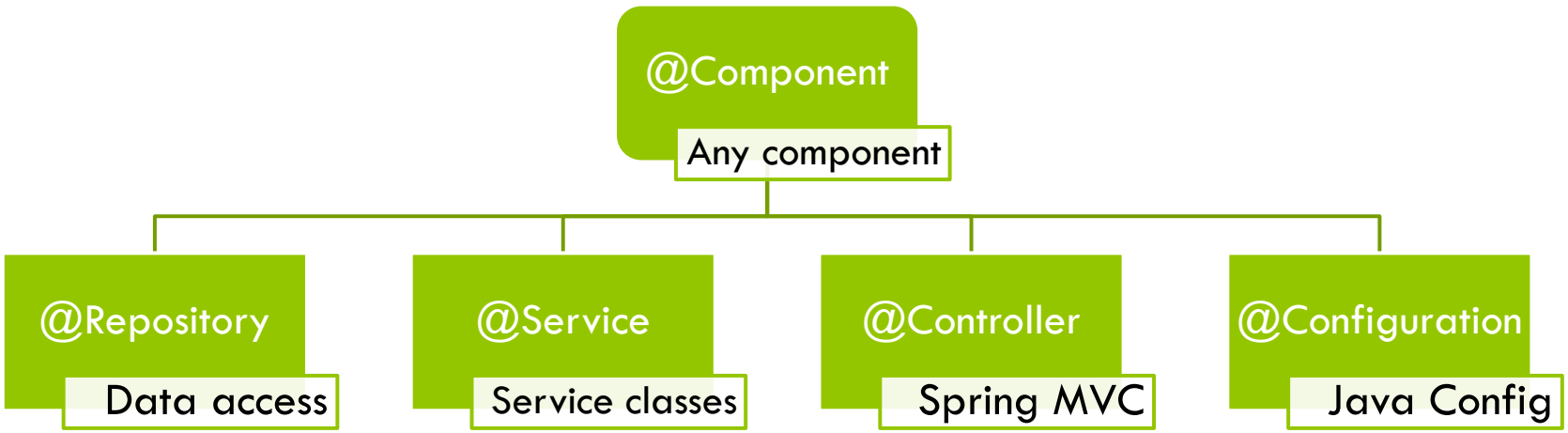
Basic Annotations

- Spring Annotations
  - `@Autowired`
  - `@Qualifier`
  - `@Required`
  - `@Value`
- JSR 250
  - `@Resource`
  - `@PostConstruct`
  - `@PreDestroy`

Basic Annotations

- context
  - `@Scope`
  - `@Bean`
  - `@DependsOn`
  - `@Lazy`
- transactional
  - `@Transactional`

Stereotypes



Basic Conguration

```
<!-- looks for annotations on beans -->
<context:annotation-config/>


<!- scan stereotyped classes and register BeanDefinition -->
<context:component-scan
    base-package="org.exhanger.repository.map,org.exhanger.api">
```

@Value

```
@Component("authenticationProvider")
public class EjbAuthenticationProvider {

    /** The provider api connection url. */
    @Value("${provider.api.url}")
    private String apiUrl;

    /** The provider api connection username. */
    private @Value("${provider.api.username}") String apiUsername;

    /** The provider api connection password. */
    @Value("${provider.api.password}")
    private String apiPassword;
}
```

@Autowired

```
@Service("accountService")
public class AccountServiceImpl {
  @Autowired
  private AccountRepository repository;

  @Autowired
  public AccountServiceImpl(AccountRepository repository) {}

  @Autowired(required = false)
  public void setRepository(AccountRepository repository) {}

  @Autowired
  public void populate(Repository r, OrderService s) {}
}
```

@Autowired

```
@Service
public class AccountServiceImpl {

  @Autowired
  private AccountRepository[] repositories;

  @Autowired
  public AccountServiceImpl(Set<AccountRepository> set) {}

  @Autowired(required = false)
  public void setRepositories(Map<String,AccountRepository> map){
  }
}
```

@Qualifier

```
public class ReportServiceImpl {
  @Autowired
  @Qualifier("main")
  private DataSource mainDataSource;

  @Autowired
  @Qualifier("freeDS")
  private DataSource freeDataSource;
}
<beans>
  <bean class="org.apache.commons.dbcp.BasicDataSource">
    <qualifier value="main"/>
  </bean>
  <bean id="freeDS" class="org.apache.commons.dbcp.BasicDataSource"/>
</beans>
```

@Scope

```
@Service
@Scope("prototype")
public class AccountServiceImpl implements AccountService {
}
```

```
@Service
@Scope(value = BeanDefinition.SCOPE_PROTOTYPE,
       proxyMode = ScopedProxyMode.TARGET_CLASS)
public class AccountServiceImpl implements AccountService {
}
```

Factory method component

```
@Component
public class CurrencyRepositoryFactory {

  @Bean
  @Lazy
  @Scope("prototype")
  @Qualifier("public")
  public CurrencyRepository getCurrencyRepository() {
    return new CurrencyMapRepository();
  }
}
```

Java-based Configuration

Standalone

- simple

```
new AnnotationConfigApplicationContext(ExchangerConfig.class);
```

- programmaticaly

```
context = new AnnotationConfigApplicationContext();
context.register(ExchangerConfig.class);
context.refresh();
```

- scanning

```
context = new AnnotationConfigApplicationContext();
context.scan("org.exchanger.config");
context.refresh();
```

Instantiate Bean

```
@Configuration
public class ExchangerRepositoryConfg {

  @Bean(name = "accountRepository", initMethod = "init")
  public AccountRepository accountRepository() {
    return new AccountMapRepository();
  }

  @Bean
  public AuditRepository auditRepository() {
    return new AuditMapRepository();
  }
}
```

@Import

```
@Configuration
@Import({RepositoryConfig.class, ServiceConfig.class})
public class ExchangerConfig {
}
```

```
@Configuration
public class RepositoryConfg {}
```

```
@Configuration
public class ServiceConfig {}
```

@ImportResource

```
@Configuration
@ImportResource("classpath:jdbc.properties")
public class ExchangerConfig {

  @Value("jdbc.url")
  private String jdbcUrl;

  @Bean
  public DataSourse dataSource() {
    return new SimpleDataSource(jdbcUrl);
  }
}
```

Dependency Injection

```
@Configuration
public class ExchangerServiceConfig {

  @Autowired
  private CurrencyRepository currencyRepository;

  public @Bean CurrencyService currencyService() {
    return new CurrencyServiceImpl(currencyRepository);
  }

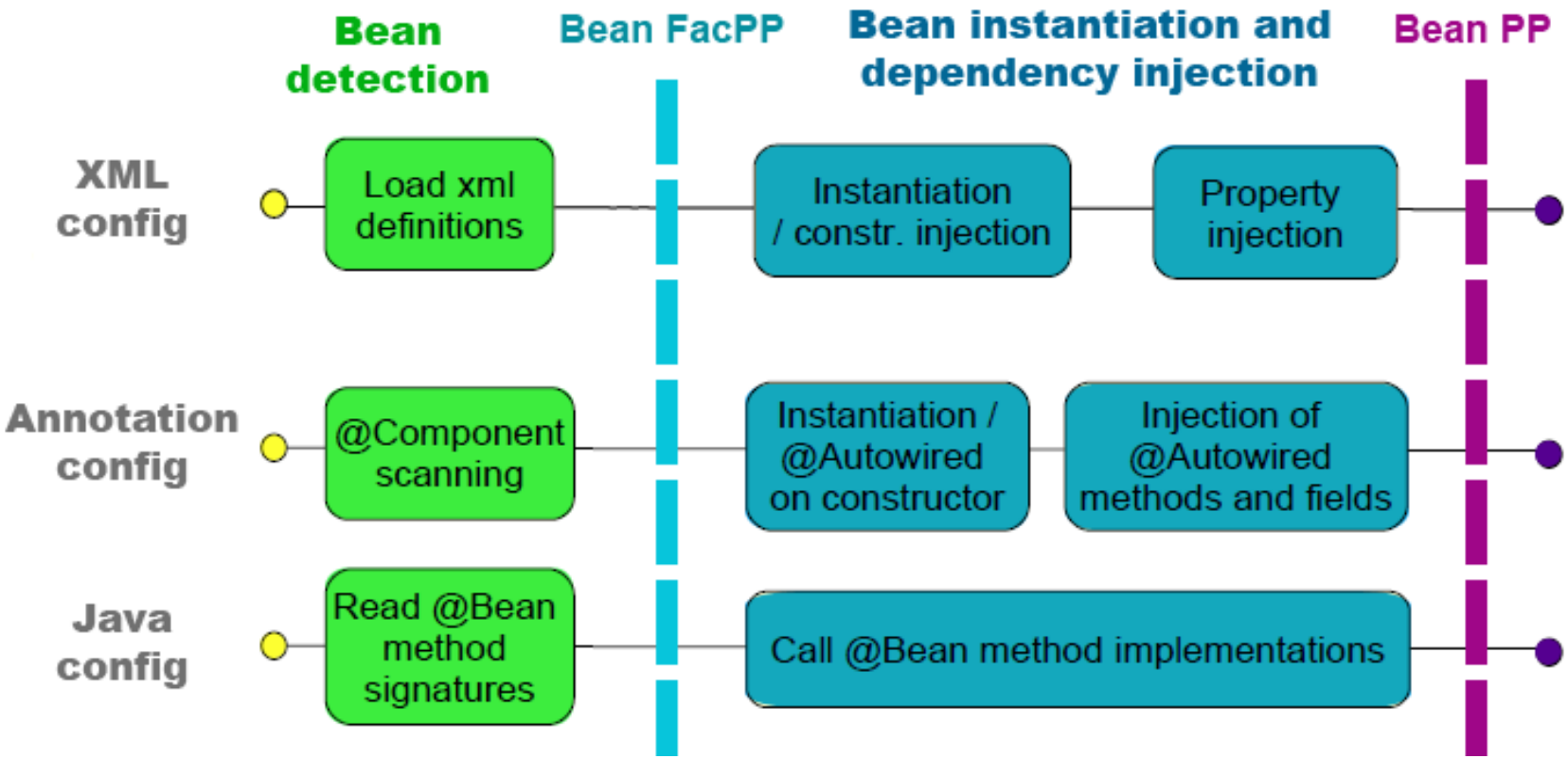
  @Bean(name = {"orderBuilder", "builder"})
  public OrderBuilder orderBuilder() {
    return new OrderBuilder(currencyService(), accountService());
  }
}
```

Summary

Benefits of IoC

- **minimizes** the amount of code
- make application more **testable**
- promote programming to **interfaces**
- loose **coupling** with minimal effort
- support **eager** instantiation and **lazy** loading
- provide control over object **lifecycle**

Context Lifecycle



Links

- site

  http://www.springsource.org/

- reference

  http://static.springsource.org/spring/docs/3.0.x/spring-framework-reference/html/

- blog

  http://blog.springsource.com/category/spring/

- forum

  http://forum.springsource.org/forumdisplay.php?f=26